

第四套题解汇总

第四套题解汇总

本题解用于整套讲评、集中查阅与打印。每道题均按照“题意整理 -> 部分分题解 -> 满分正解”的顺序编排，并保留各题已经采用的部分分梯度、最终部分分程序和满分程序。

目录

- T1 接龙
- T2 平方和
- T3 彩彩的三彩项链
- T4 表达式 (eval)

阅读说明

- 部分分按照分数从低到高讲解，并说明每一档的适用范围、复杂度和升级瓶颈。
- “最终交付的部分分程序”与对应题目的 `judge/some.cpp` 保持一致。
- 满分参考代码与对应题目的 `judge/std.cpp` 保持一致。

T1 接龙

题意整理

牛牛有 n 张牌，第 i 张牌上的数字为 a_i 。他可以任意调整这些牌的放置顺序，然后依次把牌放到桌面一列的末尾。每放置一张牌后，他可以进行至多一次收牌操作：从桌面上选择两张数字相同的牌，收走这两张牌以及它们之间的所有牌。若这两张相同的牌数字为 x ，本次共收走 len 张牌，则获得

$$x \times len$$

分。被收走的牌不会再次使用。牛牛也可以在某一回合不收牌。求把所有牌都放置完以后，能够获得的最大总分。

部分分题解

部分分总表

分值	特殊限制	做法	复杂度
20	$n \leq 100$	枚举所有牌对寻找最大重复值	$O(n^2)$
20	所有牌相同	直接输出 $a_1 n$	$O(n)$
30	$a_i \leq 1000$	小值域计数	$O(n + 1000)$
100	无额外限制	完整值域计数	$O(n + 10^5)$

小规模枚举

枚举任意两张牌，只要数字相同就更新最大的可用端点数字。得到 M 后仍然输出 $M \times n$ 。

全相等与小值域

全相等时显然可以一次收走全部牌。小值域时开一个大小为 1001 的计数数组即可。最终交付程序综合三类特殊输入，实际通过测试点 1 ~ 7，获得 70 分。

为什么不能通过全部数据

一般数据可能同时满足 $n > 100$ 、数值超过 1000 且不全相等，部分分程序没有统计完整值域。

从部分分到满分

把计数数组扩展到题目给定的 10^5 值域即可，核心上界与构造证明保持不变。

最终交付的部分分程序

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> a(n);
    int mx = 0;
    bool allSame = true;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        mx = max(mx, a[i]);
        if (i > 0 && a[i] != a[0]) {
            allSame = false;
        }
    }

    int best = 0;
    if (n <= 100) {
```

```

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                best = max(best, a[i]);
            }
        }
    }
} else if (allSame) {
    best = a[0];
} else if (mx <= 1000) {
    vector<int> cnt(1001, 0);
    for (int x : a) {
        cnt[x]++;
    }
    for (int x = 1; x <= 1000; x++) {
        if (cnt[x] >= 2) {
            best = x;
        }
    }
} else {
    cout << 0 << '\n';
    return 0;
}

cout << 1LL * best * n << '\n';
return 0;
}

```

满分正解

关键观察

设所有出现至少两次的数字中，最大的是 M 。若不存在这样的数字，则任何时刻都无法选择两个相同端点，答案为 0。

对任意一次收牌操作，其端点数字不超过 M 。若本次收走 len 张牌，得分不超过 $M \times len$ 。所有操作收走的牌互不重复，因此总分不超过

$$M \times n$$

。

另一方面，把两张数字为 M 的牌分别放在整个顺序的最前和最后，中间放入其余所有牌，最后一次收走全部 n 张牌，就能得到 $M \times n$ 。

满分算法

统计每个数字的出现次数，找到最大的、出现至少两次的数字 M ，输出 $M \times n$ 。

正确性说明

上界由每次操作的端点数字至多为 M 以及所有收走长度总和至多为 n 得到；构造两张 M 包住全部牌能够达到该上界。因此答案恰为 $M \times n$ 。

复杂度分析

时间复杂度 $O(n + V)$ ，空间复杂度 $O(V)$ ，其中 $V = 10^5$ 。

边界与易错点

- 只有出现至少两次的数字才能作为一次收牌的端点；
- 允许不收牌，所以没有重复数字时答案是 0；
- 不能只按原输入顺序计算，因为题目允许任意调整顺序；
- 结果最大为 10^{10} ，需要 long long。

参考代码

```

#include<bits/stdc++.h>
using namespace std;

```

```

const int maxv = 100000 + 10;
int cnt[maxv];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    int best = 0;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        cnt[x]++;
        if (cnt[x] >= 2) {
            best = max(best, x);
        }
    }

    cout << 1LL * best * n << '\n';
    return 0;
}

```

T2 平方和

题意整理

给定 n 个正整数。请选择两个不同位置的数字 a_i, a_j , 令

$$x = a_i^2 + a_j^2$$

。

要求 x 能被 k 整除, 并使 x 尽可能大。输入保证至少存在一组合法选择。

部分分题解

部分分总表

分值	特殊限制	做法	复杂度
20	$n \leq 2000$	枚举所有点对	$O(n^2)$
30	$k \leq 10^5$	用数组按平方余数分组	$O(n + k)$
100	无额外限制	用哈希表保存出现过的余数	期望 $O(n)$

小规模枚举

枚举 $i < j$, 直接判断平方和是否能被 k 整除。

小模数

当 $k \leq 10^5$ 时, 可以开长度为 k 的数组, 为每种平方余数保存最大的两个数字。

最终交付程序组合这两种做法, 实际通过测试点 1 ~ 5, 获得 50 分。

为什么不能通过全部数据

一般数据中 n 很大且 k 可达 10^9 : 无法枚举点对, 也不能开长度为 k 的数组。

从部分分到满分

只为实际出现过的平方余数建立哈希表, 就把空间从 $O(k)$ 降为 $O(n)$ 。

最终交付的部分分程序

```
#include<bits/stdc++.h>
using namespace std;

long long brute(const vector<long long> &a, long long k) {
    long long ans = -1;
    int n = a.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            long long value = a[i] * a[i] + a[j] * a[j];
            if (value % k == 0) {
                ans = max(ans, value);
            }
        }
    }
    return ans;
}

long long smallK(const vector<long long> &a, int k) {
    vector<pair<long long, long long>> best(k, {0, 0});
    for (long long x : a) {
        int r = (long long)((__int128)x * x % k);
        if (x > best[r].first) {
            best[r].second = best[r].first;
            best[r].first = x;
        } else if (x > best[r].second) {
            best[r].second = x;
        }
    }
}
```

```

    }
    long long ans = -1;
    for (int r = 0; r < k; r++) {
        int s = (k - r) % k;
        if (r == s) {
            if (best[r].second > 0) {
                ans = max(ans, best[r].first * best[r].first + best[r].second * best[r].second);
            }
        } else if (best[r].first > 0 && best[s].first > 0) {
            ans = max(ans, best[r].first * best[r].first + best[s].first * best[s].first);
        }
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    cin >> n >> k;
    vector<long long> a(n);
    for (long long &x : a) {
        cin >> x;
    }

    if (n <= 2000) {
        cout << brute(a, k) << '\n';
    } else if (k <= 100000) {
        cout << smallK(a, (int)k) << '\n';
    } else {
        cout << -1 << '\n';
    }
    return 0;
}

```

满分正解

关键观察

只需关心平方对 k 的余数。若

$$a_i^2 \bmod k = r,$$

那么另一项必须满足平方余数为

$$(k - r) \bmod k$$

。

对于每一种平方余数，只保留数值最大的两个元素：不同余数组合只需要各自最大值；相同余数组合需要同组中的前两大值，以保证选择不同位置。

满分算法

使用哈希表记录每个平方余数对应的最大值和次大值。扫描所有余数组，查找互补余数并更新答案。

正确性说明

任意合法点对的两个平方余数之和模 k 为 0。对于两个不同余数组，替换成各组最大值不会破坏余数条件且只会增大平方和；对于同一余数组，最优解显然由该组最大的两个位置构成。因此只保留前两大值不会遗漏答案。

复杂度分析

哈希表期望时间复杂度 $O(n)$ ，空间复杂度 $O(n)$ 。

边界与易错点

- 两个数字必须来自不同位置，因此同余数组要保存两个元素；

- 相同数值可以来自不同位置;
- 平方和最大可达 2×10^{18} , 必须使用 long long;
- 计算平方余数时使用 __int128 更稳妥。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

struct CustomHash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t seed = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + seed);
    }
};

void insertValue(pair<long long, long long> &p, long long x) {
    if (x > p.first) {
        p.second = p.first;
        p.first = x;
    } else if (x > p.second) {
        p.second = x;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    cin >> n >> k;

    unordered_map<long long, pair<long long, long long>, CustomHash> best;
    best.reserve(n * 2 + 10);
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        long long r = (long long)((__int128)x * x % k);
        auto &p = best[r];
        insertValue(p, x);
    }

    long long ans = -1;
    for (const auto &it : best) {
        long long r = it.first;
        long long s = (k - r) % k;
        auto jt = best.find(s);
        if (jt == best.end()) {
            continue;
        }
        if (r == s) {
            if (it.second.second > 0) {
                ans = max(ans, it.second.first * it.second.first + it.second.second * it.second.second);
            }
        } else {
            ans = max(ans, it.second.first * it.second.first + jt->second.first * jt->second.first);
        }
    }

    cout << ans << '\n';
    return 0;
}
```

T3 彩彩的三彩项链

题意整理

彩彩有一条首尾相连、长度为 n 的项链。每颗珠子的颜色是红、绿、蓝之一，分别用字符 r 、 g 、 b 表示。一次点击可以把一颗珠子的颜色按照以下顺序改变一次：

r -> g -> b -> r

可以多次点击同一颗珠子。

彩彩希望修改后，项链上任意两颗相邻珠子的颜色都不同。由于项链首尾相连，第 1 颗珠子和第 n 颗珠子也相邻。求最少点击次数。

部分分题解

部分分总表

分值	特殊性质	做法	复杂度
10	$n = 10$	枚举第一颜色后做三状态 DP；也可枚举全部 3^{10} 个目标串	$O(n)$ 或 $O(3^n n)$
10	$n = 1000$	三状态 DP	$O(n)$
10	只有一种颜色	三状态 DP 或分析周期着色	$O(n)$
10	只有两种颜色	三状态 DP	$O(n)$
100	无额外限制	完整线性 DP	$O(n)$

原题的前四类子任务是互相独立的特殊输入，并不是渐进复杂度分段。最终交付程序先判断是否属于这四类中的任意一类，再执行正确的三状态 DP，实际通过测试点 1 ~ 8，获得 40 分。

为什么不能通过全部数据

一般测试点长度大于 1000 且三种颜色均出现，不满足部分分程序公开承诺的任何特殊性质。

从部分分到满分

三状态 DP 本身已经能够线性处理全部长度。取消对子任务性质的限制即可成为满分程序。

最终交付的部分分程序

```
#include<bits/stdc++.h>
using namespace std;

long long solve(const string &s) {
    int n = s.size();
    const long long inf = (1LL << 60);
    long long answer = inf;

    auto id = [](char c) {
        if (c == 'r') return 0;
        if (c == 'g') return 1;
        return 2;
    };

    for (int first = 0; first < 3; first++) {
        long long dp[3] = {inf, inf, inf};
        dp[first] = (first - id(s[0]) + 3) % 3;

        for (int i = 1; i < n; i++) {
            long long ndp[3] = {inf, inf, inf};
            int original = id(s[i]);
            for (int last = 0; last < 3; last++) {
                for (int now = 0; now < 3; now++) {
                    if (last != now) {
                        ndp[now] = min(ndp[now], dp[last] + (now - original + 3) % 3);
                    }
                }
            }
            dp = ndp;
        }
        answer = min(answer, dp[first]);
    }
    return answer;
}
```

```

    }
    }
    }
    for (int c = 0; c < 3; c++) {
        dp[c] = ndp[c];
    }
}

for (int last = 0; last < 3; last++) {
    if (last != first) {
        answer = min(answer, dp[last]);
    }
}
}
return answer;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    string s;
    cin >> n >> s;

    bool have[3] = {};
    for (char c : s) {
        if (c == 'r') have[0] = true;
        if (c == 'g') have[1] = true;
        if (c == 'b') have[2] = true;
    }
    int kinds = have[0] + have[1] + have[2];

    // 覆盖 n=10、n=1000、单色和双色四类公开子任务。
    if (n == 10 || n == 1000 || kinds <= 2) {
        cout << solve(s) << '\n';
    } else {
        cout << -1 << '\n';
    }
    return 0;
}

```

满分正解

关键观察

点击颜色形成长度为 3 的循环。把 r, g, b 编码为 $0, 1, 2$ ，原颜色为 x 、目标颜色为 y 时，最少点击次数为

$$(y - x + 3) \bmod 3$$

。

项链的限制只与相邻珠子的目标颜色有关。枚举第一颗珠子的最终颜色后，就可以做线性 DP。

满分算法

对第一颗珠子的最终颜色 $first$ 枚举 $0, 1, 2$ 。定义 $dp[c]$ 为处理到当前位置且当前位置最终颜色为 c 的最小点击次数。转移时只允许前后颜色不同。处理完以后，再要求最后一颗颜色与 $first$ 不同。

正确性说明

固定第一颗目标颜色以后，任意合法方案都可以按照位置顺序分解为相邻颜色不同的转移；DP 枚举了当前目标颜色的三种可能，并对每个状态保留最小代价。最后补上首尾不同条件，就精确得到该第一颜色下的最优值。枚举三种第一颜色后取最小即为全局最优。

复杂度分析

每个位置只有常数个状态和转移，时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

边界与易错点

- 必须检查第 n 颗和第 1 颗；

- 改色是单向循环，不是任意换色一次；
- 同一颗珠子最多需要点击两次；
- n 可达 10^6 ，不需要开二维 DP。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

long long solve(const string &s) {
    int n = s.size();
    const long long inf = (1LL << 60);
    long long answer = inf;

    auto id = [](char c) {
        if (c == 'r') return 0;
        if (c == 'g') return 1;
        return 2;
    };

    for (int first = 0; first < 3; first++) {
        long long dp[3] = {inf, inf, inf};
        dp[first] = (first - id(s[0]) + 3) % 3;

        for (int i = 1; i < n; i++) {
            long long ndp[3] = {inf, inf, inf};
            int original = id(s[i]);
            for (int last = 0; last < 3; last++) {
                for (int now = 0; now < 3; now++) {
                    if (last != now) {
                        ndp[now] = min(ndp[now], dp[last] + (now - original + 3) % 3);
                    }
                }
            }
            for (int c = 0; c < 3; c++) {
                dp[c] = ndp[c];
            }
        }

        for (int last = 0; last < 3; last++) {
            if (last != first) {
                answer = min(answer, dp[last]);
            }
        }
    }
    return answer;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    string s;
    cin >> n >> s;
    cout << solve(s) << '\n';
    return 0;
}
```

T4 表达式 (eval)

题意整理

给定一个只由正整数、加号 + 和乘号 * 组成的合法表达式 S 。

你可以在表达式中的任意合法位置添加任意多对括号，但不能改变数字和运算符的原有顺序。求能够得到的最大表达式值。

部分分题解

部分分总表

测试点	特殊限制	可行做法	分值
1 ~ 2	$ S \leq 10$	枚举括号或区间 DP	10
3 ~ 6	所有数不超过 9	单字符解析并按加法段计算	20
7 ~ 10	只有加法	求所有数之和	20
11 ~ 12	只有乘法	求所有数之积	10
13 ~ 14	$ S \leq 1000$	使用大整数线性解析	10
15 ~ 20	一般数据	完整大整数线性解析	30

原题前 14 个测试点由若干互相独立的特殊性质构成。最终交付程序识别小长度、小数字、单一运算符或 $|S| \leq 1000$ 的输入，再使用正确的大整数计算，实际获得 70 分。

为什么不能通过全部数据

最后六个测试点同时包含加法和乘法、长度超过 1000，并含大于 9 的多位整数，不满足任何部分分支。

从部分分到满分

完整算法本身仍是“加法段求和后相乘”，只需取消对子任务性质的限制，并对所有长度不超过 5000 的表达式统一使用任意精度整数。

最终交付的部分分程序

```
#include<bits/stdc++.h>
#include<boost/multiprecision/cpp_int.hpp>
using namespace std;

boost::multiprecision::cpp_int readNumber(const string &s, int l, int r) {
    boost::multiprecision::cpp_int x = 0;
    for (int i = l; i < r; i++) {
        x = x * 10 + (s[i] - '0');
    }
    return x;
}

boost::multiprecision::cpp_int solveExpression(const string &s) {
    using boost::multiprecision::cpp_int;
    cpp_int product = 1;
    cpp_int currentSum = 0;
    int n = s.size();
    int i = 0;

    while (i < n) {
        int j = i;
        while (j < n && isdigit((unsigned char)s[j])) {
            j++;
        }
        currentSum += readNumber(s, i, j);
        if (j == n) {
            break;
        }
        if (s[j] == '*') {
            product *= currentSum;
            currentSum = 0;
        }
        i = j + 1;
    }
    return product * currentSum;
}
```

```

    }
    return product * currentSum;
}

int main() {
    freopen("eval.in", "r", stdin);
    freopen("eval.out", "w", stdout);

    string s;
    cin >> s;

    bool allPlus = true;
    bool allMultiply = true;
    bool allSmall = true;
    int i = 0;
    while (i < (int)s.size()) {
        int j = i;
        boost::multiprecision::cpp_int value = 0;
        while (j < (int)s.size() && isdigit((unsigned char)s[j])) {
            value = value * 10 + (s[j] - '0');
            j++;
        }
        if (value > 9) {
            allSmall = false;
        }
        if (j < (int)s.size()) {
            if (s[j] != '+') allPlus = false;
            if (s[j] != '*') allMultiply = false;
        }
        i = j + 1;
    }

    // 覆盖原题测试点 1~14 的小长度、小数字、单一运算符等子任务。
    if ((int)s.size() <= 1000 || allSmall || allPlus || allMultiply) {
        cout << solveExpression(s) << '\n';
    } else {
        cout << 0 << '\n';
    }
    return 0;
}

```

满分正解

关键观察

把表达式按照乘号 * 分成若干段，每一段内部全部由加号连接。由于所有数都是正数，利用分配律，把每一段的和作为一个乘法因子，能让跨越加号两侧的正数尽可能相乘。

例如

$2 \times 3 + 4 \times 5 + 6$

最大值为

$2 \times (3 + 4) \times (5 + 6)$ 。

因此答案就是：每个最大加法段先求和，再把所有段的和相乘。

正确性说明

考虑表达式中的任意一个乘号。通过在它两侧不断应用分配律，乘号能够与相邻加法段中的所有正项相乘；正数条件保证增加这些乘法组合不会使结果变小。最终可得到所有由乘号分隔的最大加法段之和的乘积。反过来，这个式子可以通过合法添加括号实现，因此既是上界也是可达值。

满分算法

线性解析每个正整数：遇到 + 就继续累加当前段，遇到 * 就把当前段和乘入答案并清零。使用 `boost::multiprecision::cpp_int` 保存任意精度整数。

复杂度分析

忽略大整数位运算代价，扫描复杂度为 $O(|S|)$ ，空间复杂度与答案位数同阶。

边界与易错点

- 不能使用 long long 保存输入数字或答案;
- 连续解析多位整数时不能按单个字符处理;
- 最后一段需要在循环结束后乘入;
- 原样例解释中的 9876 是笔误, 应为 9867。

参考代码

```
#include<bits/stdc++.h>
#include<boost/multiprecision/cpp_int.hpp>
using namespace std;

boost::multiprecision::cpp_int readNumber(const string &s, int l, int r) {
    boost::multiprecision::cpp_int x = 0;
    for (int i = l; i < r; i++) {
        x = x * 10 + (s[i] - '0');
    }
    return x;
}

boost::multiprecision::cpp_int solveExpression(const string &s) {
    using boost::multiprecision::cpp_int;
    cpp_int product = 1;
    cpp_int currentSum = 0;
    int n = s.size();
    int i = 0;

    while (i < n) {
        int j = i;
        while (j < n && isdigit((unsigned char)s[j])) {
            j++;
        }
        currentSum += readNumber(s, i, j);
        if (j == n) {
            break;
        }
        if (s[j] == '*') {
            product *= currentSum;
            currentSum = 0;
        }
        i = j + 1;
    }
    return product * currentSum;
}

int main() {
    freopen("eval.in", "r", stdin);
    freopen("eval.out", "w", stdout);

    string s;
    cin >> s;
    cout << solveExpression(s) << '\n';
    return 0;
}
```